

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

PEFT, LoRA y QLoRA en la Práctica

Módulo 8 · Ingeniería de Modelos y LLMOps

Implementación con Hugging Face PEFT y TRL, hiperparámetros clave, evaluación y plataformas de fine-tuning como servicio.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

PEFT, LoRA y QLoRA en la Práctica

En 2021, el paper "LoRA: Low-Rank Adaptation of Large Language Models" (Hu et al.) transformó el fine-tuning de LLMs. La idea central: en lugar de actualizar todos los parámetros del modelo (costoso en memoria y cómputo), añadir matrices de bajo rango entrenables a las capas de atención. El resultado: fine-tuning de modelos de 70B parámetros en una sola GPU A100, con calidad similar al fine-tuning completo. En 2023, QLoRA (Dettmers et al.) lo llevó más lejos combinando LoRA con cuantización NF4, reduciendo la VRAM necesaria a un cuarto.

PEFT, LoRA, y QLoRA son hoy el estándar industrial para adaptar modelos LLM grandes a casos de uso específicos en hardware accesible. Este tema cubre la implementación práctica: cómo configurar LoRA, cómo elegir los hiperparámetros correctos, cómo evitar los problemas más comunes, y cómo evaluar que el fine-tuning ha producido las mejoras esperadas sin degradar las capacidades generales del modelo.

La anatomía de LoRA y QLoRA

LoRA: adaptadores de bajo rango

LoRA congela todos los pesos originales del modelo y añade dos matrices pequeñas (A y B) a cada capa de atención seleccionada. Para una capa con pesos W de dimensiones $d \times k$, LoRA añade $\Delta W = A \times B$ donde A tiene dimensiones $d \times r$ y B tiene dimensiones $r \times k$, con r (el rank) mucho menor que d o k (típicamente $r=4, 8, 16, 32$). Solo A y B se entrenan, reduciendo el número de parámetros entrenables de millones a miles. Para Llama 3 8B, el fine-tuning completo requiere actualizar 8.000 millones de parámetros; LoRA con $r=16$ aplicado a las matrices de atención requiere actualizar ~4 millones de parámetros (0.05% del total).

QLoRA: LoRA + cuantización NF4

QLoRA combina LoRA con cuantización NF4 (Normal Float 4-bit) del modelo base. El modelo base se carga en 4 bits (reduciendo su VRAM en 4x respecto a FP16), y los adaptadores LoRA se entrenan en BF16 (precisión completa). Durante el forward pass, el modelo base se desquantiza dinámicamente a BF16 para los cálculos. El resultado: fine-tuning de Llama 3 70B (que requeriría 140GB VRAM en FP16) en una sola GPU A100 80GB. La pérdida de calidad respecto al fine-tuning en FP16 es inferior al 1% para la mayoría de las tareas.

Módulos objetivo: ¿qué capas adaptar con LoRA?

LoRA se aplica típicamente a las matrices de proyección de atención: q_proj , k_proj , v_proj , o_proj (los componentes de la atención multi-cabeza). Opcionalmente, se puede aplicar también a las matrices del FFN ($gate_proj$, up_proj , $down_proj$). Aplicar LoRA solo a las matrices de atención (el default conservador) es suficiente para la mayoría de los casos de fine-tuning. Aplicarlo también al FFN aumenta la capacidad de adaptación a cambio de más parámetros entrenables y mayor riesgo de sobreajuste con datasets pequeños.

Implementación con Hugging Face PEFT y TRL

El stack estándar para fine-tuning con QLoRA es: transformers + peft + bitsandbytes + trl (Transformer Reinforcement Learning). El proceso: cargar el modelo base en NF4 (BitsAndBytesConfig con load_in_4bit=True y bnb_4bit_compute_dtype=bfloat16), preparar para k-bit training (prepare_model_for_kbit_training), configurar LoRA (LoraConfig con r, lora_alpha, target_modules, lora_dropout), obtener el modelo PEFT (get_peft_model), y entrenar con SFTTrainer de TRL que gestiona el formato del dataset de instrucciones.

Los hiperparámetros de entrenamiento más importantes: learning_rate=2e-4 (para LoRA, no para el modelo completo), per_device_train_batch_size=2-4 (limitado por VRAM), gradient_accumulation_steps=4-8 (para simular batches más grandes), num_train_epochs=1-3 (raramente más de 3 evita sobreajuste), warmup_ratio=0.03, lr_scheduler_type="cosine". La tasa de aprendizaje más alta que en fine-tuning completo (1e-4 vs 5e-5) es apropiada porque solo los adaptadores pequeños se actualizan.

Guardar y cargar adaptadores LoRA

Los adaptadores LoRA se guardan separados del modelo base: model.save_pretrained("lora_adapter"). Esto produce archivos mucho más pequeños que el modelo completo: los adaptadores LoRA de un Llama 3 8B con r=16 ocupan ~50-200MB vs los 16GB del modelo en FP16. Para inferencia, se carga el modelo base y se aplican los adaptadores: model = PeftModel.from_pretrained(base_model, "lora_adapter"). También es posible fusionar los adaptadores en los pesos base (model.merge_and_unload()) para eliminar la latencia adicional de los adaptadores durante la inferencia, a cambio de no poder descargarlos.

SECCIÓN 4 · EJEMPLOS REALES

- Fine-tuning de Llama 3.1 8B con QLoRA para extracción de datos de facturas (startup fintech): 3.000 pares de factura → JSON estructurado. QLoRA (r=16, targets q_proj/v_proj) en una A100 40GB, 3 horas de entrenamiento. Resultado: accuracy de extracción del 94% vs 81% del modelo base con prompting. Tamaño del adaptador: 67MB. Coste del entrenamiento: ~\$30 en cloud.
- Fine-tuning de Mistral 7B para generación de código en estilo interno (empresa de software): 8.000 ejemplos de código Python siguiendo las convenciones internas de la empresa. QLoRA (r=32, incluye gate_proj/up_proj/down_proj del FFN además de atención) para capturar el estilo de código. El modelo generado sigue automáticamente las convenciones sin instrucciones adicionales, reduciendo el tiempo de revisión de código.
- Multi-LoRA serving (empresa con múltiples clientes): el mismo modelo base Llama 3 8B sirve múltiples clientes, cada uno con su propio adaptador LoRA fine-tuneado. vLLM soporta multi-LoRA serving: carga el modelo base una vez y aplica dinámicamente el adaptador correcto para cada request, eliminando la necesidad de instancias separadas por cliente.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar rank demasiado alto sin necesitarlo. r=16 es suficiente para la mayoría de los casos. r=64 o r=128 aumenta significativamente los parámetros entrenables y el

riesgo de sobreajuste con datasets pequeños, sin mejora proporcional en calidad para tareas estándar. Empieza con $r=8$ o $r=16$ y escala solo si los resultados son insuficientes.

Error 2: No usar gradient checkpointing. Sin gradient checkpointing, el fine-tuning con QLoRA de modelos grandes puede superar la VRAM disponible aunque el modelo quepa en memoria. Activa `gradient_checkpointing=True` en el `TrainingArguments`: reduce el uso de VRAM en 4-8x a cambio de ~33% más tiempo de entrenamiento.

Error 3: Dataset de fine-tuning con instrucciones inconsistentes. Si el dataset de fine-tuning tiene el 60% de ejemplos con un formato de respuesta y el 40% con otro, el modelo aprenderá comportamiento inconsistente. La curaduría del dataset (revisar manualmente una muestra aleatoria del 10%) es el paso más importante para garantizar la calidad del fine-tuning.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El setup mínimo para QLoRA con TRL en Python: `pip install transformers peft bitsandbytes trl accelerate`. Para el training: `import SFTTrainer from trl`, carga el modelo base con `BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_quant_type="nf4")`, configura `LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"], lora_dropout=0.05)`, y entrena con `SFTTrainer` con el dataset en formato alpaca o chat. Hugging Face tiene guías completas en hf.co/docs/trl para el formato correcto del dataset.

Para evaluar el fine-tuning: divide el dataset en 80% train / 10% validation / 10% test. Monitoriza la pérdida de validación durante el entrenamiento (WandB o TensorBoard). Para cuando la validación deja de mejorar (early stopping). Evalúa el test set con las métricas de tu caso de uso. Evalúa también en un subset de MMLU para detectar catastrophic forgetting. Compara con el modelo base + el mejor prompt para verificar que el fine-tuning mejora sobre el baseline.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T6 (Fine-tuning de modelos): introduce el concepto de fine-tuning con LoRA; este tema profundiza en la implementación práctica.
- M4-T7 (Entrenamiento a gran escala): QLoRA con DeepSpeed para fine-tuning distribuido multi-GPU.
- M8-T2 (Fine-tuning vs prompting): el contexto de cuándo aplicar LoRA/QLoRA.

SECCIÓN 8 · RESUMEN EJECUTIVO

LoRA añade matrices entrenables de bajo rango ($r=4-32$) a las capas de atención, reduciendo los parámetros entrenables al 0.05-1% del total con calidad similar al fine-tuning completo. QLoRA combina LoRA con cuantización NF4 del modelo base, reduciendo la VRAM necesaria 4x. Stack estándar: `transformers + peft + bitsandbytes + trl`. Hiperparámetros clave: $r=8-16$ (el más importante), `lora_alpha=2r`, `target_modules=[q_proj, v_proj]`, `lr=2e-4`, `gradient_checkpointing=True`. Los adaptadores se guardan separados del modelo base (~50-200MB). Multi-LoRA serving en vLLM: múltiples adaptadores sobre el mismo modelo base. La curaduría del dataset es más importante que los hiperparámetros de entrenamiento.